

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

**Лабораторна робота №7
з дисципліни
«Основи розробки програмного забезпечення мовою
програмування Python»
на тему
“Основи обробки винятків у мові програмування Python”**

Виконав:
студент групи ІМ-42
Жила Іван Дмитрович
номер варіанту: 8

Перевірив:
Фещенко К. Ю

Київ 2026

Мета роботи

Ознайомитися з механізмами обробки виняткових ситуацій у мові програмування Python. Набути практичних навичок використання конструкцій *try-except*, *else*, *finally*, а також створення та застосування користувацьких винятків для забезпечення надійності та коректності програм.

Короткі теоретичні відомості:

1. Поняття помилки та винятку

Під час виконання програм можуть виникати різноманітні помилки, зумовлені некоректним введенням даних користувачем, арифметичними помилками, відсутністю необхідних файлів або неправильними типами змінних. У мові Python такі помилки називаються винятками (exceptions).

Виняток - це спеціальна подія, яка перериває нормальний хід виконання програми. Якщо виняток не обробити, програма завершує роботу аварійно. Коректна обробка винятків дозволяє запобігти збоям та підвищити стійкість програмного коду.

2. Структура блоку обробки винятків

Для керування винятковими ситуаціями в Python використовується вбудований механізм на основі конструкції *try-except*. Повна структура має такий вигляд:

- *try* - містить фрагмент коду, в якому потенційно може виникнути виняток.
- *except* - містить код для обробки конкретного типу помилки. Програма може мати кілька *blocks except* для диференційованої реакції на різні помилки.
- *else* - виконується лише тоді, коли в блоці *try* не виникло жодного винятку. Це дозволяє логічно відокремити коректний сценарій від обробки помилок.
- *finally* - виконується обов'язково і завжди, незалежно від того, виник виняток чи ні. Зазвичай використовується для закриття файлів та звільнення ресурсів.

3. Стандартні винятки Python

Вбудовані класи винятків допомагають точно визначити причину помилки. Найбільш поширеними є:

- *ValueError* - виникає, коли тип значення правильний, але сам вміст некоректний (наприклад, спроба перетворити текстовий рядок у число).

- `TypeError` - фіксується при спробі виконати операцію з несумісними типами даних.
- `ZeroDivisionError` - виникає під час спроби ділення на нуль.
- `FileNotFoundError` - сигналізує про те, що потрібний файл відсутній або недоступний.
- `KeyError` / `IndexError` - виникають у разі звернення до неіснуючого ключа словника або індексу списку відповідно.

4. Користувацькі винятки

Якщо стандартних класів недостатньо для опису специфічних помилок конкретної задачі, Python дозволяє створювати користувацькі винятки. Для цього розробляють новий клас, який обов'язково успадковується від базового класу `Exception`. Це підвищує зрозумілість логіки програми, особливо в об'єктно-орієнтованих системах.

Загальне завдання:

Розробити програму, яка виконує дії, згідно індивідуального варіанту. Передбачити можливість виникнення та реалізувати коректну обробку винятків. Інформацію про виникнення винятків необхідно виводити на екран та зберігати у логфайл.

Індивідуальне завдання - Написати програму для обробки арифметичних помилок із записом інформації про виняток у файл. (8 варіант)

Текст програми:

```
# Лабораторна 7

# Тема: Основи обробки винятків у мові програмування Python

# Загальне завдання: Передбачити можливість виникнення та
# реалізувати коректну обробку винятків.
# Інформацію про виникнення винятків необхідно виводити на
# екран та зберігати у логфайл.
```

Варіант 8: Написати програму для обробки арифметичних помилок із записом інформації про виняток у файл.

```
import os
import datetime
```

Функція для відображення вмісту файлу в консолі (для перевірки)

```
def print_file_content(filename, description):
    print(f"\nВміст файлу: {description} ({filename})")
    if os.path.exists(filename):
        with open(filename, 'r', encoding='utf-8') as f:
            print(f.read())
    else:
        print("Файл не знайдено.")
    print("-" * 50)
```

Функція для запису помилок у лог-файл

```
def log_error_to_file(filename, error_type, message):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(filename, 'a', encoding='utf-8') as f:
        f.write(f"[{timestamp}] ТИП: {error_type} | ПОВІДОМЛЕННЯ: {message}\n")
```

Функція для виконання завдання з обробки помилок введення (ValueError)

```
def process_input_task(log_file):
    print("\nПеревірка введення даних")
    try:
        # Спроба зчитати та перетворити введені дані у число
        user_input = input("Введіть ціле число: ")
        number = int(user_input)
    except ValueError as e:
        # Обробка помилки некоректного типу даних
        error_msg = f"Неможливо перетворити '{user_input}' у ціле число."
        print(f"Перехоплено виняток: {error_msg}")
        log_error_to_file(log_file, "ValueError", error_msg)
    else:
```

```

        # Виконується, якщо помилок не було
        print(f"Успішно. Ви ввели число: {number}")
    finally:
        # Виконується завжди
        print("Блок перевірки введення завершив роботу.")

# Функція для виконання завдання з обробки помилок роботи з
# файлами (FileNotFoundError)
def process_file_task(filename, log_file):
    print("\nЧитання неіснуючого файлу")
    try:
        # Спроба відкрити файл, якого немає
        with open(filename, 'r', encoding='utf-8') as f:
            content = f.read()
    except FileNotFoundError as e:
        # Обробка помилки відсутності файлу
        error_msg = f"Файл '{filename}' не знайдено в системі."
        print(f"Перехоплено виняток: {error_msg}")
        log_error_to_file(log_file, "FileNotFoundError", error_msg)
    else:
        print("Файл успішно прочитано.")
    finally:
        print("Блок роботи з файлом завершив роботу.")

# Функція для виконання індивідуального завдання
# Обробка арифметичних помилок (ZeroDivisionError та
# OverflowError)
def process_individual_task(log_file):
    print("\nАрифметичні помилки")

    # Ділення на нуль
    try:
        print("Спроба виконати ділення на нуль (10 / 0)")
        result = 10 / 0
    except ZeroDivisionError as e:
        error_msg = "Спроба ділення на нуль."
        print(f"Помилка: {error_msg}")
        log_error_to_file(log_file, "ZeroDivisionError", error_msg)

    # Переповнення при обчисленні великих чисел
    try:
        print("Спроба обчислити занадто велике число")

```

```

(експонента)")
    # Швидке зведення у величезний ступінь викликає
помилку переповнення float
    result = 2.0 ** 1000000
    except OverflowError as e:
        error_msg = "Результат арифметичної операції занадто
великий (переповнення)."
        print(f"Помилка: {error_msg}")
        log_error_to_file(log_file, "OverflowError", error_msg)
    else:
        print(f"Результат обчислення: {result}")
    finally:
        print("Обробка арифметичних помилок завершена.")

def main():
    log_file = 'errors_log.txt'

    # Очищаємо або створюємо новий лог-файл на початку
роботи програми
    with open(log_file, 'w', encoding='utf-8') as f:
        f.write("ЛОГ ЗАПУСКУ ПРОГРАМИ\n")

    # Обробка помилок введення
    process_input_task(log_file)

    # Обробка помилок роботи з файлами
    process_file_task('non_existent_file.txt', log_file)

    # Обробка арифметичних помилок
    process_individual_task(log_file)

    # Виведення фінального лог-файлу для перевірки
результатів
    print_file_content(log_file, "Журнал винятків (Лог-файл)")

if __name__ == "__main__":
    main()

```

Скріншоти:

Перевірка введення даних

Введіть ціле число: 5.5

Перехоплено виняток: Неможливо перетворити '5.5' у ціле число.

Блок перевірки введення завершив роботу.

Читання неіснуючого файлу

Перехоплено виняток: Файл 'non_existent_file.txt' не знайдено в системі.

Блок роботи з файлом завершив роботу.

нАрифметичні помилки

Спроба виконати ділення на нуль (10 / 0)

Помилка: Спроба ділення на нуль.

Спроба обчислити занадто велике число (експонента)

Помилка: Результат арифметичної операції занадто великий (переповнення).

Обробка арифметичних помилок завершена.

Вміст файлу: Журнал винятків (Лог-файл) (errors_log.txt)

ЛОГ ЗАПУСКУ ПРОГРАМИ

[2026-05-18 21:27:50] ТИП: ValueError | ПОВІДОМЛЕННЯ: Неможливо перетворити '5.5' у ціле число.

[2026-05-18 21:27:50] ТИП: FileNotFoundError | ПОВІДОМЛЕННЯ: Файл 'non_existent_file.txt' не знайдено в системі.

[2026-05-18 21:27:50] ТИП: ZeroDivisionError | ПОВІДОМЛЕННЯ: Спроба ділення на нуль.

[2026-05-18 21:27:50] ТИП: OverflowError | ПОВІДОМЛЕННЯ: Результат арифметичної операції занадто великий (переповнення).

Перевірка введення даних

Введіть ціле число: 7

Успішно. Ви ввели число: 7

Блок перевірки введення завершив роботу.

Читання неіснуючого файлу

Перехоплено виняток: Файл 'non_existent_file.txt' не знайдено в системі.

Блок роботи з файлом завершив роботу.

нАрифметичні помилки

Спроба виконати ділення на нуль (10 / 0)

Помилка: Спроба ділення на нуль.

Спроба обчислити занадто велике число (експонента)

Помилка: Результат арифметичної операції занадто великий (переповнення).

Обробка арифметичних помилок завершена.

Вміст файлу: Журнал винятків (Лог-файл) (errors_log.txt)

ЛОГ ЗАПУСКУ ПРОГРАМИ

[2026-05-18 21:28:45] ТИП: FileNotFoundError | ПОВІДОМЛЕННЯ: Файл 'non_existent_file.txt' не знайдено в системі.

[2026-05-18 21:28:45] ТИП: ZeroDivisionError | ПОВІДОМЛЕННЯ: Спроба ділення на нуль.

[2026-05-18 21:28:45] ТИП: OverflowError | ПОВІДОМЛЕННЯ: Результат арифметичної операції занадто великий (переповнення).

Висновок:

Під час виконання лабораторної роботи я ознайомився з механізмами обробки виняткових ситуацій у мові програмування Python. Набув практичних навичок застосування конструкцій `try-except`, `else` та `finally` для підвищення стійкості, надійності та коректності програмного коду.

У ході виконання завдань я реалізував комплексну систему захисту програми від критичних збоїв:

- Обробку помилок введення (`ValueError`) та роботи з файлами (`FileNotFoundError`), що дозволило програмі не "вилітати" у разі некоректних дій користувача або відсутності ресурсів.
- Індивідуальне завдання, у межах якого налагодив перехоплення арифметичних помилок (ділення на нуль `ZeroDivisionError` та переповнення `OverflowError`).
- Логування винятків, забезпечивши виведення інформації про помилки на екран та її автоматичне документування у текстовий лог-файл із точним часом їх виникнення.

Тестування програми з різними вхідними даними (коректним числом 7 та некоректним 5.5) підтвердило, що блок `finally` чітко виконується за будь-яких умов. Створений застосунок продемонстрував високу стабільність, логічну завершеність та готовність до обробки непередбачуваних сценаріїв роботи.